

DEVELOPER English ✨

➔ বাংলাদেশের ডেভেলপারদের জন্য
GLOBAL হওয়ার ভাষা



অধ্যায় এক

এই বইটা কেন তোমার জন্য

একটা চেনা দৃশ্যের কথা দিয়ে শুরু করা যাক। তুমি কোড লিখতে পারো। প্রজেক্ট বানাতে পারো। কোনো সমস্যা সামনে পড়লে সমাধান খুঁজে বের করতে পারো। তোমার দক্ষতা নিয়ে কোনো প্রশ্ন নেই। কিন্তু যেদিন কেউ সামনে বসে জিজ্ঞেস করে, "Explain your project" সেদিন হঠাৎ সব থমকে যায়। মাথা ফাঁকা হয়ে যায়। কথা আসে না।

এই একটা মুহূর্ত বাংলাদেশের হাজার হাজার মেধাবী ডেভেলপারের সামনে একটা অদৃশ্য দেয়াল হয়ে দাঁড়িয়ে থাকে। সবচেয়ে বড় কথা হলো, এই সমস্যাটাকে কেউ কখনো সিরিয়াসলি নেয় না। অথচ এই একটা সমস্যাই অনেকের career-কে local market-এর গণ্ডির মধ্যে আটকে রাখে।

সমস্যাটা আসলে কোথায়?

অনেকে মনে করেন সমস্যাটা তাঁদের ইংরেজির দুর্বলতায়। এই ধারণাটা আসলে ঠিক নয়। সমস্যাটা ইংরেজি জানা বা না-জানার নয়, সমস্যাটা হলো ইংরেজিকে কখনো একটা tool হিসেবে ব্যবহার না করা।

তুমি Git ব্যবহার করো, VS Code ব্যবহার করো, Stack Overflow ব্যবহার করো। এগুলো তোমার কাজের হাতিয়ার। ইংরেজিও ঠিক তেমনি একটি হাতিয়ার। এটি কোনো আলাদা পরীক্ষার বিষয় নয়, এটি একটি skill যা ব্যবহারের মাধ্যমে তৈরি হয়।

দুটি গল্পের পার্থক্য

একটি interview-এর কথা বলা যাক। দুজন candidate, প্রায় একই মানের কোডিং দক্ষতা। দুজনকেই একই প্রশ্ন করা হলো, "Explain your project."

প্রথমজন বললেন:

I make app... user can add something...

দ্বিতীয়জন বললেন:

I built a task management app using React. It allows users to create, update, and delete tasks. I also implemented local storage for data persistence.

ফলাফল? প্রথমজন reject, দ্বিতীয়জন selected। পার্থক্য skill-এ ছিল না, পার্থক্য ছিল expression-এ।

একজন তাঁর কাজকে সুন্দরভাবে উপস্থাপন করতে পারলেন, আরেকজন পারলেন না।

" ভালো developer হওয়া যথেষ্ট নয়। Global developer হতে হলে communication লাগবে "

এই বইটা সেই communication-টাই তোমার কাছে পৌঁছে দেওয়ার জন্য লেখা হয়েছে। এখানে grammar শেখানো হবে না। শেখানো হবে কীভাবে real situation-এ নিজেকে প্রকাশ করতে হয়, কীভাবে নিজের কাজ explain করতে হয়, এবং কীভাবে সেই ভাষাটা প্রতিদিন ব্যবহার করতে হয়।

তুমি পিছিয়ে নেই। তুমি weak নও। তুমি শুধু এখনো ঠিকভাবে train হওনি। সেটাই আজ থেকে বদলাবে।

অধ্যায় দুই

Developer English System

একটা সহজ প্রশ্ন দিয়ে শুরু করা যাক। তুমি কি আগে কখনো ইংরেজি improve করার চেষ্টা করেছিলে? Grammar book খুলেছিলে, vocabulary মুখস্থ করার চেষ্টা করেছিলে, YouTube video দেখেছিলে। কিন্তু সপ্তাহ দুয়েক পরেই সব থেমে গেছে। এই ঘটনাটা কেন বারবার ঘটে? কারণ একটাই। কোনো system ছিল না।

Random Learning বনাম System Learning

বেশিরভাগ মানুষ ইংরেজি শেখার সময় এলোমেলোভাবে এগোয়। আজ grammar, কাল vocabulary, পরশু speaking। কোনো structure নেই, কোনো flow নেই। তুমি কি এভাবে code লেখো? আজ function, কাল API, পরশু database, কোনো structure ছাড়া? না। তাহলে ইংরেজির ক্ষেত্রেও একই নিয়ম প্রযোজ্য।

এই কারণেই আমরা একটি নির্দিষ্ট system নিয়ে কাজ করব, যার নাম Developer English System।

Developer English System

1. System-এর চারটি স্তর

এই system চারটি জিনিসের উপর দাঁড়িয়ে আছে। প্রথমত Reading, অর্থাৎ সঠিক জায়গা থেকে সঠিকভাবে পড়া। দ্বিতীয়ত Writing, অর্থাৎ প্রতিদিন লেখার অভ্যাস তৈরি করা। তৃতীয়ত Speaking, অর্থাৎ মুখে বলার practice করা। এবং চতুর্থত Problem Explaining, অর্থাৎ নিজের কাজ গুছিয়ে বোঝানো।

এই চারটির বাইরে কিছু দরকার নেই। Grammar ধীরে ধীরে আসবে। Vocabulary নিজে নিজে তৈরি হবে, কারণ তুমি ব্যবহার করবে।

2. Input ও Output-এর পার্থক্য

Reading ও Listening হলো input। Writing ও Speaking হলো output। বেশিরভাগ মানুষ শুধু input নেয়, video দেখে, article পড়ে। কিন্তু output দেয় না। ফলে তারা বুঝতে পারে, কিন্তু বলতে পারে না।

Developer হিসেবে এটা বিপজ্জনক। কারণ তোমাকে শুধু বুঝলেই হবে না, explain করতে হবে। এই কারণেই output practice এই system-এর কেন্দ্রে।

3. Daily Routine: মাত্র ৩০ মিনিট

সময়ের অভাব একটি common অজুহাত। কিন্তু এই system-এ দরকার মাত্র তিরিশ মিনিট। দশ মিনিট পড়া, দশ মিনিট লেখা, দশ মিনিট কথা বলা। এই তিরিশ মিনিট যদি তুমি প্রতিদিন দাও, ষাট দিনের মধ্যে noticeable পরিবর্তন আসবে।

তবে একা practice করলে একটা সমস্যা থেকে যায়। ভুল হলে নিজে নিজে ঠিক করা যায় না। এই জায়গাতেই AI একটা বড় ভূমিকা রাখে। AI তোমার teacher, partner এবং corrector তিনটাই হতে পারে। পরের অধ্যায়ে আমরা সেটাই দেখব।

অধ্যায় তিন

AI Practice System

একটা বাস্তব সমস্যার কথা বলা যাক। তুমি English practice করতে চাও, কিন্তু করবে কার সাথে? বন্ধুরা busy। Teacher-এর কাছে যেতে সময় ও অর্থ দুটোই লাগে। Regularity বজায় রাখাও কঠিন। এই কারণেই অনেকে practice শুরুই করতে পারেন না।

এই সমস্যার একটা সমাধান এখন তোমার হাতের মুঠোয় আছে এবং এটা আগে কখনো এত সহজলভ্য ছিল না। AI-এর সাহায্যে তুমি এখন চব্বিশ ঘণ্টা practice করতে পারো, কাউকে disturb না করে, কারো উপর depend না করে।

সঠিকভাবে AI ব্যবহার করা

সমস্যা হলো বেশিরভাগ মানুষ AI ঠিকভাবে ব্যবহার করেন না। তাঁরা random প্রশ্ন করেন, random উত্তর পান এবং কোনো improvement হয় না। কারণ কোনো system নেই। এই chapter-এ আমরা সেই system তৈরি করব, যার নাম AI Practice System।

চারটি ধাপ

এই system চারটি ধাপে কাজ করে।

ধাপ এক: Write

আজকের coding সমস্যা বা যেকোনো technical বিষয় ইংরেজিতে লিখো। ভুল হলেও সমস্যা নেই। Perfect হওয়া নয়, শুরু করাটাই এখানে আসল কাজ।

I am facing an error while installing Node.js. It shows permission denied.

ধাপ দুই: Correct

এই লেখাটা AI-কে দাও। নিচের prompt ব্যবহার করো:

Act as an English mentor for software developers. Correct my sentence and improve clarity.

AI তোমার লেখা ঠিক করে দেবে।

ধাপ তিন: Rewrite

Corrected version দেখে নিজে আবার লিখো। এই ধাপটি skip করলে কোনো লাভ হবে না। Improvement হয় rewriting-এ, শুধু পড়লে হয় না।

ধাপ চার: Speak

এখন sentence টি জোরে বলো। প্রথমদিকে awkward লাগবে, এটাই স্বাভাবিক। কিন্তু এই awkwardness-এর মধ্য দিয়েই confidence তৈরি হয়।

...

Real-World Use Cases

GitHub Commit:

Write: I fixed bug in login. Correct: I fixed a bug in the login functionality.

Stack Overflow:

Write: I have problem with API. It not working. Correct: I am facing an issue with an API. It is not working.

Interview Preparation:

Question: Tell me about your project. Write answer > AI correct > Rewrite > Speak

Write > Correct > Rewrite > Speak

এই loop-টিই তোমার daily practice। ত্রিশ দিন টানা করো। তুমি নিজেই পার্থক্য বুঝবে। AI তোমাকে help করবে, কিন্তু practice তোমাকেই করতে হবে। Shortcut নেই, system আছে।

অধ্যায় চার

Reading ও Writing Optimization

একটা সত্যি কথা বলা যাক। তুমি হয়তো অনেক কিছু পড়ো, article, documentation, tutorial। কিন্তু এই পড়াগুলো কি তোমার ইংরেজি improve করছে? সত্যি উত্তর বেশিরভাগ ক্ষেত্রেই না। কারণটা সহজ। তুমি passive ভাবে পড়ো। চোখ দিয়ে পড়ো, কিন্তু মাথা দিয়ে process করো না।

Passive বনাম Active Reading

Reading দুইভাবে করা যায়। Passive reading মানে স্ক্রল করা, দেখা, এগিয়ে যাওয়া। Active reading মানে বোঝা, ভাঙা, ব্যবহার করা। Developer হিসেবে তোমার দরকার active reading।

সঠিক source বেছে নেওয়া এখানে গুরুত্বপূর্ণ। Random blog পড়লে লাভ কম। Dev.to, Medium-এর developer articles, GitHub README এবং official documentation এগুলো real-world language, তাই এগুলো থেকেই শুরু করো।

পড়ার সময় ধীর হওয়া দরকার। দ্রুত পড়া smart নয়, বুঝে পড়াটাই smart। একটা line পড়ো, থামো, নিজেকে জিজ্ঞেস করো, "আমি কি এটা explain করতে পারব?" যদি না পারো, তাহলে তুমি আসলে বোঝোনি।

Sentence ভাঙার অভ্যাস

একটা উদাহরণ দেওয়া যাক।

The application uses REST API to fetch data.

এটাকে ভাঙো। Application মানে app, uses মানে ব্যবহার করে, fetch data মানে data নেওয়া। এখন নিজের ভাষায় বলো। এটাই শেখা।

AI-কে smartly ব্যবহার করো। AI-কে বলো "Explain this sentence in simple English." AI simplified version দেবে। কিন্তু এখানেই থামলে হবে না। সবচেয়ে critical ধাপ হলো নিজে আবার লেখা। এখানেই input output-এ রূপান্তরিত হয়।

Writing: ভয়টাই সবচেয়ে বড় বাধা

বেশিরভাগ মানুষ writing avoid করেন কারণ মনে হয় ভুল হবে। কিন্তু এই ভয়টাই growth-কে থামিয়ে দেয়। Writing improve করার একটাই rule:

Write badly, but write.

Perfect হওয়ার দরকার নেই। Consistent হওয়া দরকার। প্রতিদিন তিনটি কাজ করো। আজ যা পড়লে তা থেকে তিনটি line নিজের মতো করে লিখো। আজকের coding সমস্যা ইংরেজিতে লিখো। এবং যেকোনো একটা feature explain করো, যেমন:

This feature allows users to login using email and password.

Professional English-এর আসল মানে

অনেকে মনে করেন fancy English লিখতে হবে। Complex sentence, hard words এগুলো আসলে confusion তৈরি করে। Developer English-এর নিয়ম হলো Simple, Clear এবং Direct। এই তিনটি গুণ থাকলেই সেটা professional।

Reading তোমাকে input দেয়। Writing তোমাকে output দেয়। কিন্তু improvement হয় output-এ। তাই শুধু পড়লেই হবে না, লিখতে হবে।

অধ্যায় পাঁচ

Speaking ও Interview Communication

একটা চেনা অনুভূতির কথা বলা যাক। ইংরেজি sentence মাথায় ঠিকঠাক সাজিয়ে নিতে পারো, কিন্তু মুখে বলতে গেলে আটকে যাও। মনে হয় কথাগুলো গলায় আটকে আছে। এটা খুব সাধারণ একটা সমস্যা।

কারণটাও সহজ। তুমি practice করো মাথার ভেতরে, কিন্তু real world-এ লাগে মুখ দিয়ে। এই gap-টাই তোমাকে পিছিয়ে দেয়। Speaking skill build না করলে, তুমি যতই জানো, তুমি দেখাতে পারবে না।

Confidence বনাম Perfection

Speaking improve করার জন্য perfect English দরকার নেই। দরকার confidence। আর confidence আসে practice থেকে। AI-কে conversation partner হিসেবে ব্যবহার করো। এই prompt দিয়ে শুরু করো:

Talk with me about programming. Ask simple questions. Correct my mistakes.

AI জিজ্ঞেস করবে, "What are you learning now?" তুমি বলো, "I am learning React. I am building a project." ভুল হলেও সমস্যা নেই। Correction হবে।

Short Sentence Rule

অনেকে বড় sentence বানাতে গিয়ে আটকে যান। তোমার rule হবে, ছোট, পরিষ্কার এবং সরল।

I am learning React. I am building a project. I am trying to understand hooks.

এই level যথেষ্ট। এটাই শুরু।

Interview: আসলে কী দেখা হয়?

Interview হলো একটি communication test। Interviewer তোমার ইংরেজি judge করেন না, তাঁরা দেখেন তুমি explain করতে পারো কিনা। Common প্রশ্নগুলোর জন্য আগের loop-ই ব্যবহার করো।

একটা উদাহরণ দেওয়া যাক। "Tell me about your project" প্রশ্নের উত্তরে প্রথমে লিখো "I made a todo app. It has add and delete." তারপর AI দিয়ে correct করাও। Corrected version হবে "I built a todo application where users can add and delete tasks." এরপর নিজে rewrite করো এবং জোরে বলো।

নিজের Project ৩ লাইনে Explain করে

এই structure follow করো। প্রথম লাইনে বলো what you built, দ্বিতীয় লাইনে বলো what it does, তৃতীয় লাইনে বলো tech used।

I built a task management app. It allows users to add and manage tasks. I used React and Firebase.

এটা বারবার practice করো। এটাই তোমার foundation।

• • •

"আমি ভুল বললে মানুষ হাসবে" এই চিন্তা বাদ দাও। Real world-এ মানুষ clarity দেখে, confidence দেখে। Grammar নয়। Speaking avoid করলে তুমি stuck থাকবে। Speaking শুরু করলে তুমি grow করবে।

অধ্যায় ছয়

Real-World Developer Communication

ইংরেজি শেখার আসল জায়গা classroom নয়, YouTube নয়। Real world। যেখানে তুমি প্রতিদিন কাজ করো, GitHub, StackOverflow, client chat, team discussion, এই জায়গাগুলোতেই তোমার ইংরেজি সত্যিকার অর্থে তৈরি হবে।

GitHub: শুধু Code নয়, Communication-ও

GitHub শুধু code রাখার জায়গা নয়। এটা একটা communication platform। তুমি এখানে commit message, issue এবং pull request লিখো। এগুলো ঠিকভাবে লিখতে পারাটা একটা গুরুত্বপূর্ণ skill।
তুলনা করো:

ভুল: fixed bug ঠিক: Fix login validation issue causing incorrect password rejection

ভালো commit message-এ তিনটি জিনিস থাকে। কী fix হয়েছে, কোথায় সমস্যা ছিল এবং কী impact ছিল। Rule টা মনে রাখো:

Action + Area + Result |

Bug Report লেখার সঠিক উপায়

ভুল: Login not working.

ঠিক: I found a bug in the login function. When I click the login button, nothing happens.
Steps to reproduce: 1. Open login page 2. Enter email and password 3. Click login

Rule: Problem > Context > Steps |

StackOverflow: প্রশ্ন করার শিল্প

Vague প্রশ্নের উত্তর কেউ দেয় না। তোমার প্রশ্ন হবে specific।

ভুল: My code not working, Help.

ঠিক: I am trying to install Node.js but I get this error: [error message] What could be the reason and how can I fix it?

Rule: What you did > What happened > What you expect I

Client Communication

Client তোমার code দেখে না, তোমার communication দেখে।
তিনটি উত্তর তুলনা করে:

Client: Can you fix this issue?

দুর্বল: Yes. I try.

ভালো: Yes, I will check the issue and update you soon.

সেরা: I will review the issue and get back to you within a few hours.

পার্থক্যটা লক্ষ্য করো। Clarity, confidence এবং professional tone। এই তিনটির সমন্বয়েই একজন developer professional হয়ে ওঠেন।

Team Communication

Team-এ কাজ করলে short, clear, direct এই তিনটি গুণ সবচেয়ে জরুরি।

I am working on the login feature. I will complete it by today. I am facing an issue with the API. I need some help.

• • •

তত্ত্ব তোমাকে prepare করে। Use তোমাকে transform করে।

আজ থেকেই শুরু করো। একটা commit message লিখো, একটা bug report লিখো, AI দিয়ে check করো, rewrite করো। তুমি যত বেশি real world-এ ইংরেজি ব্যবহার করবে, তত দ্রুত improve করবে।

শেষ অধ্যায়

Mindset ও Transformation

এখন পর্যন্ত তুমি একটি পূর্ণাঙ্গ system শিখেছো। AI Practice, Reading, Writing, Speaking এবং Real-world Communication। কিন্তু একটাই প্রশ্ন বাকি থাকে। তুমি কি আসলেই change করবে?

সবচেয়ে বড় বাধা

সমস্যা কখনো knowledge নয়। সমস্যা হলো execution। অনেকেই জানেন কী করতে হবে, কিন্তু করেন না। তুমি যদি এই বই পড়ে কিছু না করো, তাহলে ছয় মাস পরেও তুমি একই জায়গায় থাকবে। কোড লিখবে কিন্তু explain করতে পারবে না। Interview দেবে কিন্তু স্পষ্টভাবে বলতে পারবে না। এটাই harsh reality।

দুটো পথ

তুমি এখন দুটো পথের সামনে দাঁড়িয়ে আছো। প্রথম পথ হলো এই বই পড়া, মাথা নাড়া, তারপর কিছুই করো না। ফলাফল? No change। দ্বিতীয় পথ হলো এই বইয়ের system follow করো, daily practice করো, ভুল করো, ঠিক করো। ফলাফল? Transformation। Decision তোমার।

Identity Shift

"আমি ইংরেজি পারি না" এই বিশ্বাসটা বাদ দাও। নতুন বিশ্বাস হোক: "আমি ইংরেজি ব্যবহার করতে শিখছি।" এই একটা shift তোমাকে সামনে এগিয়ে নিয়ে যাবে।

আরও একটা জরুরি কথা। তুমি শুধু learner নও। তুমি developer। Developer যারা problem solve করে, build করে, improve করে। তোমার নতুন পরিচয় হলো:

Developer who communicates in English

এই পরিচয় আজ থেকে accept করো।

৯০ দিনের Roadmap

প্রতিদিন মাত্র ত্রিশ মিনিট। দশ মিনিট reading, দশ মিনিট writing এবং দশ মিনিট AI speaking। ত্রিশ দিন করো, result দেখবে। ষাট দিন করো, confidence build হবে। নব্বই দিন করো, তুমি নিজেই অবাক হবে। শর্ত একটাই। Consistency। Motivation নয়, Consistency।

• • •

ছয় মাস পর তুমি কোথায় থাকতে চাও?

Local market-এ stuck? নাকি global opportunity-তে?

উত্তর তোমার। এখন একটাই কাজ। তোমার শেষ coding problem-টা ইংরেজিতে লিখো। এটাই তোমার প্রথম পদক্ষেপ।

শুরু করো। এখনই।